

NPC- Problema de Coloración de Grafos

Thomas Jaramillo Aguirre - Juan Jose Valencia

July 2026

1 Introduction

El problema de coloración de grafos consiste en asignar colores a los vértices de un grafo de manera que dos vértices adyacentes (unidos por una arista), no compartan el mismo color. Dependiendo de la formulación del problema, el objetivo puede ser determinar si es posible colorear el grafo utilizando un número determinado de colores o encontrar el número mínimo de colores necesarios para obtener una coloración válida.

1.1 Problema de decision

Dado un grafo no dirigido ($G = (V, E)$) y un entero positivo (k), el problema de decisión de coloración de grafos pregunta si existe una función de coloración:

$$c : V \rightarrow 1, 2, \dots, k$$

tal que para toda arista $((u, v) \in E)$, se cumpla que:

$$c(u) \neq c(v)$$

En otras palabras, se debe determinar si los vértices del grafo pueden colorearse usando a lo sumo (k) colores, sin que dos vértices conectados por una arista compartan el mismo color.

2 Ejemplo

Considérese el siguiente grafo:

$$V = \{A, B, C, D\}$$

$$E = \{(A, B), (B, C), (C, D), (D, A), (A, C)\}$$

donde V representa el conjunto de vértices y E el conjunto de aristas.

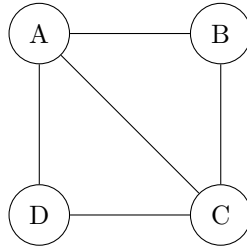


Figure 1: Grafo de entrada.

El grafo de entrada se muestra en la Figura 1.

Se desea determinar si el grafo puede colorearse utilizando tres colores, denominados C_1 , C_2 y C_3 , de manera que dos vértices adyacentes no compartan el mismo color.

Una coloración válida es la siguiente:

Vértice	Color
A	C_1
B	C_2
C	C_3
D	C_2

La solución obtenida puede visualizarse en la Figura 2.

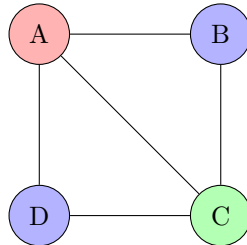


Figure 2: Coloración válida del grafo utilizando tres colores.

Se observa que todos los vértices unidos por una arista poseen colores diferentes, por lo que la coloración es válida. Además, los vértices A , B y C forman un triángulo, lo que implica que al menos tres colores son necesarios para colorear el grafo. En consecuencia, el número cromático del grafo es

$$\chi(G) = 3.$$

3 Demostración

Un problema A es NP-completo si cumple las siguientes dos condiciones:

- A es NP -difícil, es decir, todo problema NP puede reducirse en tiempo polinómico a A , en particular se puede demostrar haciendo la respectiva reducción de un problema NP -completo (NP -difícil) al problema A dado.
- $A \in NP$, es decir, toda solución de A se puede verificar en tiempo polinómico.

Para la posterior demostración se utilizara el NP -completo NAE-3-SAT, donde "NAESAT significa Not All Equal SAT. Es decir, en cada clausula del problema 3-SAT, no todos sus literales son falsos o no todos son verdaderos. En otras palabras, debe haber por lo menos un literal falso y por lo menos uno verdadero" Castillo Ungar, 2023

3.1 Reducción

Entonces para la demostración de que la Coloración de Grafos es NP -completo primero hagamos la reducción de un problema NP -completo a la Coloración de Grafos.

Primeramente reduzcamos 3-SAT a 3-Coloración. Con esta idea primero representaremos las variables de la formula con los nodos y sus valores de verdad seran representados por colores. Ahora supongamos que hay 4 variables x, y, z, w , para las que corresponden 9 nodos iniciales, con lo que el grafo se veria de la siguiente manera:

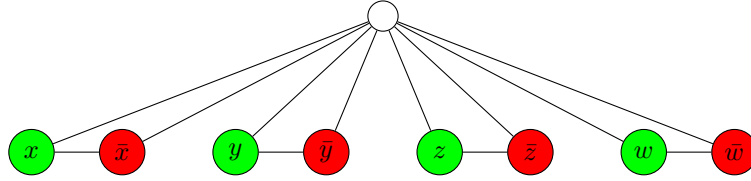


Figure 3: Graph coloring widget for variable assignments.

Ahora se debe encontrar un gadget el cual al conectar tres variables este es 3 coloreable si y solo si por lo menos una de las tres variables es verdadera. El problema con esto es que cualquier grafo que sea 3 coloreable cuando se conecte a cuatro vertices verdes, también será coloreable cuando se conecte a cuatro vertices rojos. Entonces escogeremos un tipo de fórmula el cual no tenga este problema, es decir NAE-3-SAT.

Para esto, debemos encontrar un gadget talque sea 3-coloreable si y solo si no todos los vértices a los que está coenctado son del mismo color, Este gadget es un triángulo. tomando como clausulas de nuestra fórmula a las siguientes expresiones $\bar{x} \vee y \vee z$, $y \vee \bar{z} \vee x$ y $x \vee z \vee \bar{w}$, entonces el grafo se veria como en la figura 4.

Manteniendo este algoritmo para cualquier fórmula NAE-3-SAT, podemos construir un grafo de tamaño $3n+2m+1$ donde n es la cantidad de cláusulas y m es la cantidad de literales.

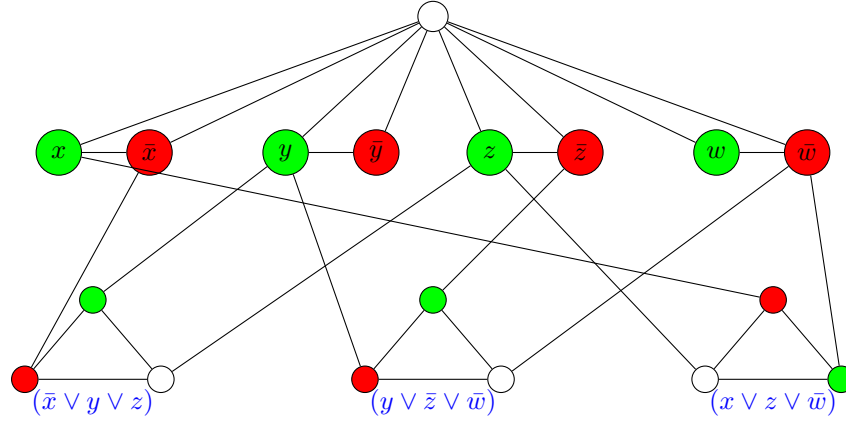


Figure 4: Full graph coloring reduction with clause gadgets.

Así, $\text{NAE-3-SAT} \leq 3\text{-Coloreable}$, luego la 3-colorabilidad es NP-completo.

La preservación del "Sí" y del "No", en esta reducción consiste también en el buen uso del NAE-3-SAT, antes que nada se tiene en cuenta la equivalencia entre valor-color. Así el valor verdadero es equivalente al verde, el valor falso al Rojo y el blanco será para el nodo raíz o neutral, todos los valores básicos estarán conectados a la raíz como indica 3. veamos primeramente la conservación del valor para instancias del "Sí":

- Escogamos una instancia NAE-satisfacible, esta debe tener al menos un valor distinto sean entonces los colores verde, verde y rojo. Solo se pueden escoger colores distintos para los vértices conectados a cada uno de estos, dado de que el gadget escogido es un triángulo se requieren 3 colores distintos para cada vértice. Gracias a la distribución verde, verde y rojo, tenemos la libertad de escoger Blanco, Rojo y Verde, para cada respectiva conexión con el grafo

Posteriormente veamos que conserva el "No":

- Si una cláusula es no NAE-Satisfacible entonces los tres valores son equivalentes, supongamos que se evalúan como Verde. Así todos los vértices del gadget (triángulo) están conectados a nodos verdes, con lo anterior solo podemos pintar los nodos del gadget de color Blanco y Rojo, lo anterior plantea un absurdo para el coloreado de un grafo ya que para un grafo tipo triángulo se tiene que no se puede colorear con menos de tres colores, ya que al pintarlo al menos una arista conectará dos vértices de un mismo color

3.2 Algoritmo de verificabilidad

la lógica planteada en el algoritmo consiste en primeramente verificar que el número de coloración de la instancia dada no supere al requerido y posteriormente recorrer la lista de aristas comprobando que no exista un arista donde sus dos vértices tengan el mismo color, con esto planteado el pseudocódigo para el algoritmo sería el de 1, se debe tener en cuenta que para el algoritmo es indistinto referenciar a los colores por su nombre o por una clasificación distinta como lo puede ser la asignación de números en el orden natural.

Algorithm 1 Verificación del Coloreo de un Grafo

```
1: procedure COLORING_VER( $V, E, N$ )
2:           // Entrada:  $V$  = Diccionario que mapea cada vértice con su color respectivo
3:                               //  $E$  = Lista de todas las aristas, con la forma (u,v)
4:                               //  $N$  = Máximo del número de colores permitido
5:           // Salida: True si el coloreo es valido y el número de colores es menor o
               igual a  $N$ , False en otro caso
6:           // Paso 1: Validar que los colores estén en el rango permitido [1,  $N$ ]
7:           for  $v \in V$  do
8:               if  $V[v] > N$  then
9:                   return False                                // El color excede el límite máximo
10:              end if
11:           end for
12:           // Paso 2: Validar que no haya vértices vecinos con el mismo color
13:           for  $e \in E$  do
14:               if  $V[e[0]] == V[e[1]]$  then
15:                   return False                                // Conflicto detectado entre nodos adyacentes
16:               end if
17:           end for
18:           return True                                          // El coloreo de la instancia es válido
19: end procedure
```

A partir del análisis del anterior algoritmo presentado, se puede deducir que su complejidad es de orden polinomial -específicamente lineal con respecto al tamaño del grafo-. La estructura consiste en dos bucles secuenciales independientes. El primero recorre el conjunto de vértices, así requiriendo un tiempo proporcional a $|V|$. Posteriormente, el segundo ciclo hace una iteración sobre la lista de aristas, lo que toma un tiempo equivalente a $|E|$. Al no existir anidamiento entre ambos bloques, sus tiempos de ejecución se combinan mediante la adición siendo así finalmente su complejidad asintótica de la forma $O(|V| + |E|)$

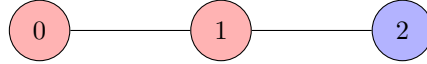
4 Casos de prueba

Para validar el algoritmo de verificabilidad se implementaron cinco casos de prueba. Estos casos permiten comprobar si el algoritmo identifica correctamente coloraciones válidas, coloraciones inválidas y situaciones donde la asignación de colores está incompleta.

4.1 Caso 1: Coloreado incorrecto

$$V = \{0 : C_0, 1 : C_0, 2 : C_1\}$$

$$E = \{(0, 1), (1, 2)\}$$

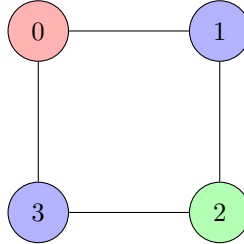


El resultado esperado es **False**, ya que los vértices 0 y 1 son adyacentes y tienen el mismo color C_0 . Por tanto, la coloración no es válida.

4.2 Caso 2: Exceso de colores en un grafo cuadrado

$$V = \{0 : C_0, 1 : C_1, 2 : C_2, 3 : C_1\}$$

$$E = \{(0, 1), (1, 2), (2, 3), (3, 0)\}, \quad n = 2$$

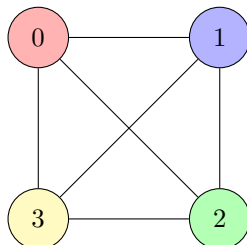


El resultado esperado es **False**. Aunque el grafo cuadrado puede colorearse correctamente utilizando únicamente dos colores, en este caso el vértice 2 tiene asignado el color C_2 . Como el parámetro $n = 2$, únicamente se permiten los colores C_0 y C_1 . Por esta razón, el algoritmo debe determinar que la coloración es inválida, ya que se está utilizando un color fuera del rango permitido.

4.3 Caso 3: Grafo completo K_4 válido

$$V = \{0 : C_0, 1 : C_1, 2 : C_2, 3 : C_3\}$$

$$E = \{(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)\}$$

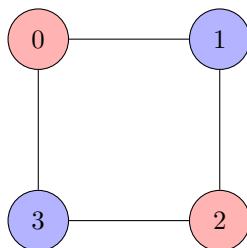


El resultado esperado es **True**. Este grafo corresponde al grafo completo K_4 , donde todos los vértices están conectados entre sí. Como cada vértice tiene un color diferente y $n = 4$, la coloración es válida.

4.4 Caso 4: Grafo bipartito C_4 válido

$$V = \{0 : C_0, 1 : C_1, 2 : C_0, 3 : C_1\}$$

$$E = \{(0, 1), (1, 2), (2, 3), (3, 0)\}$$



El resultado esperado es **True**. Este grafo es un ciclo de cuatro vértices, C_4 , y puede colorearse con dos colores porque es bipartito. Ningún par de vértices adyacentes comparte el mismo color.

4.5 Caso 5: Coloreado incompleto

$$V = \{0 : C_0\}$$

$$E = \{(0, 1)\}$$



El resultado esperado es `KeyError`. Esto ocurre porque el vértice 1 aparece en el conjunto de aristas, pero no tiene un color asignado en el diccionario V . Por tanto, al intentar acceder al color de dicho vértice, el algoritmo genera una excepción.

5 Bibliografía

References

Castillo Ungar, J. (2023, May). *Problemas NP-completos en grafos* [Proyecto de grado (Matemático)]. Universidad de los Andes.